

Mini Project

TITLE: Implement Non-Serial Polyadic Dynamic Programming with GPU Parallelization

AIM: To design and implement **Non-Serial Polyadic Dynamic Programming (Matrix Chain Multiplication)** using **GPU parallelization with CUDA**, and analyze the performance benefits of parallel execution.

PRE-REQUISITES:

1. Knowledge of **Dynamic Programming (DP)**
2. Understanding of **Matrix Multiplication and its computational cost**
3. Knowledge of **Parallel Computing concepts (SIMD, threads, GPU)**
4. Basic understanding of **CUDA architecture (Kernel, Threads, Blocks)**

THEORY:-

Introduction to Dynamic Programming

Dynamic Programming is an optimization technique used to solve complex problems by breaking them into smaller overlapping subproblems and storing their results to avoid redundant computations. It is particularly useful for problems that exhibit optimal substructure and overlapping subproblems. In such problems, the optimal solution of the whole problem depends on the optimal solutions of its subproblems. By storing intermediate results, Dynamic Programming reduces time complexity compared to naive recursive approaches.

Matrix Chain Multiplication Problem

Matrix Chain Multiplication is a classical Dynamic Programming problem in which we are given a sequence of matrices $A_1, A_2, A_3, \dots, A_n$. Each matrix A_i has dimensions:

$$A_i = d_i \times d_{i+1}$$

The goal is to compute the product $A_1 \times A_2 \times \dots \times A_n$ such that the total number of scalar multiplications is minimized. Although the multiplication order of matrices cannot be changed, the way they are grouped (parenthesized) can be altered to achieve optimal performance.

Cost of Matrix Multiplication

The cost of multiplying two matrices depends on their dimensions. If a matrix of size $p \times q$ is multiplied with a matrix of size $q \times r$, then the number of scalar multiplications required is:

$$\text{Cost} = p \times q \times r$$

This formula is fundamental in determining the total cost of a particular parenthesization. Different groupings result in different intermediate matrix sizes, which leads to variation in total computation cost.

Importance of Parenthesization

Although matrix multiplication is associative, meaning:

$$(A \times B) \times C = A \times (B \times C)$$

the number of operations required is not the same for both cases. This is because the dimensions of intermediate matrices differ depending on how the matrices are grouped. Therefore, finding the optimal parenthesization is necessary to minimize the total number of scalar multiplications and improve computational efficiency.

Dynamic Programming Formulation

To efficiently solve the Matrix Chain Multiplication problem, we define a DP table where:

$$dp[i][j] = \text{Minimum cost of multiplying matrices from } i \text{ to } j$$

The recurrence relation used is:

$$dp[i][j] = \min_{i \leq k < j} (dp[i][k] + dp[k+1][j] + d_i \cdot d_{k+1} \cdot d_{j+1})$$

Here, k represents the index at which the matrix chain is split. The expression consists of:

- Cost of multiplying left subchain: $dp[i][k]$
- Cost of multiplying right subchain: $dp[k+1][j]$
- Cost of multiplying the resulting matrices: $d_i \cdot d_{k+1} \cdot d_{j+1}$

DP Table Construction

The DP table is constructed using a bottom-up approach. Initially, the diagonal elements are set as:

$$dp[i][i] = 0$$

since a single matrix does not require multiplication. The table is then filled diagonally by increasing the chain length. For each chain length L , all possible values of i and $j = i + L$ are considered, and the optimal cost is computed using the recurrence relation. This ensures that all smaller subproblems are solved before being used in larger ones.

Time and Space Complexity

The time complexity of the standard Dynamic Programming solution is:

$$O(n^3)$$

because of three nested loops: one for chain length, one for starting index i , and one for partition index k . The space complexity is:

$$O(n^2)$$

due to the storage required for the DP table. While this approach is efficient compared to naive methods, it becomes computationally expensive for large values of n .

Need for Parallelization

To overcome the limitations of the sequential approach, parallelization is introduced. The key observation is that all DP entries corresponding to the same chain length are independent and can be computed simultaneously. This allows multiple computations to be executed in parallel, significantly reducing execution time. Parallel computing is especially beneficial for large-scale problems where the number of computations is very high.

Introduction to CUDA

CUDA (Compute Unified Device Architecture) is a parallel computing platform that enables the use of GPU resources for general-purpose computing. GPUs contain a large number of cores that can perform computations simultaneously. In CUDA, functions known as kernels are executed by multiple threads in parallel. Threads are grouped into blocks, and blocks form a grid, enabling efficient large-scale parallel execution.

Parallelization Strategy in MCM

In Matrix Chain Multiplication, parallelization is achieved by assigning each DP table entry $dp[i][j]$ (for a fixed chain length) to a separate thread. Since all entries on the same diagonal depend only on previously computed values, they can be calculated independently. This diagonal-based computation allows efficient utilization of GPU threads and ensures that multiple DP entries are computed simultaneously.

CUDA-Based Implementation Concept

In the CUDA implementation, memory is allocated using unified memory so that both CPU and GPU can access it. The kernel function computes each DP entry using the recurrence relation:

$$\text{cost} = dp[i][k] + dp[k+1][j] + d_i \cdot d_{k+1} \cdot d_{j+1}$$

Each thread evaluates all possible values of k and stores the minimum cost. After each kernel execution, synchronization is performed to ensure that all threads have completed their computations before proceeding to the next chain length.

Advantages of Parallel Dynamic Programming

Parallel Dynamic Programming using CUDA significantly reduces computation time by exploiting the parallel processing capabilities of GPUs. It allows efficient handling of large datasets and demonstrates how classical algorithms can be adapted for high-performance computing environments. This approach is widely applicable in fields such as scientific computing, optimization problems, and large-scale data processing.

Algorithm

1. Define the dimension array

$$\text{dims}[] = \{d_0, d_1, \dots, d_n\}$$

where matrix $A_i = d_i \times d_{i+1}$.

2. Allocate memory for DP table $dp[n][n]$ and dimension array using CUDA unified memory.
3. Initialize DP table:

$$dp[i][i] = 0 \text{ for all } i$$

4. For chain length $len = 1$ to $n - 1$, repeat:
 - a. Launch CUDA kernel with n threads.
 - b. Each thread computes one element $dp[i][j]$, where:
$$j = i + len$$

- c. For each possible split $k = i$ to $j - 1$, compute:
$$\text{cost} = dp[i][k] + dp[k + 1][j] + d_i \cdot d_{k+1} \cdot d_{j+1}$$

- d. Store minimum cost:

$$dp[i][j] = \min(\text{cost})$$

- e. Synchronize using `cudaDeviceSynchronize()`.

5. After completion, the result is:

$$dp[0][n - 1]$$

6. Print the minimum multiplication cost and free allocated memory.

Conclusion

The Matrix Chain Multiplication problem was efficiently solved using Dynamic Programming with CUDA-based parallelization. This approach reduces computation time by exploiting parallelism and improves performance for large inputs.

```
In []: # Steps to Setup Collab NoteBook for CUDA Programming

# In Menu Bar Click on Runtime -> Change Runtime type -> Select T4 GPU

# Then execute the below command to get real-time information about your GPU and CUDA
environment
```

```
In [17]: !nvidia-smi
```

Tue Mar 24 12:04:06 2026

```
+-----+
| NVIDIA-SMI 580.82.07                Driver Version: 580.82.07          CUDA Version: 13.0     |
+-----+-----+-----+-----+-----+-----+
| GPU  Name            Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |
+=====+=====+=====+=====+=====+=====+
|   0   Tesla T4               Off   | 00000000:00:04:0 Off |                    0 |
| N/A   62C    P8              20W / 70W |  0MiB / 15360MiB |      0%      Default |
|                                           | N/A             |
+-----+-----+-----+-----+-----+-----+
|
+-----+-----+-----+-----+-----+-----+
| Processes:
| GPU   GI    CI          PID    Type    Process name                        GPU Memory
|       ID    ID                                   |       Usage
+=====+=====+=====+=====+=====+=====+
| No running processes found
+-----+-----+-----+-----+-----+-----+
|
```

```
In []: # Below cell contains the code and the command to write the program to your CUDA file

# '%writefile nspdp.cu' this command has keyword 'writefile' which means the content below
this command must be written to the specified CUDA file

# The command is not necessary you can create a seperate file with .cu extension and type
your code in it
```

```

In [18]: %%writefile nspdp.cu
#include <stdio.h>
#include <limits.h>
#include <cuda_runtime.h>

#define N 4

__global__ void computeDP(int *dp, int *dims, int n, int len) {
    int i = threadIdx.x;

    if (i < n - len) {
        int j = i + len;

        int minCost = INT_MAX;

        for (int k = i; k < j; k++) {
            int cost = dp[i*n + k] + dp[(k+1)*n + j] +
                dims[i]*dims[k+1]*dims[j+1];

            if (cost < minCost)
                minCost = cost;
        }

        dp[i*n + j] = minCost;
    }
}

int main() {
    int dims[] = {10, 20, 30, 40, 30};
    int n = N;

    int *dp;
    int *d_dims;

    cudaMallocManaged(&dp, n*n*sizeof(int));
    cudaMallocManaged(&d_dims, (n+1)*sizeof(int));

    // Copy dims
    for (int i = 0; i < n+1; i++)
        d_dims[i] = dims[i];

    // Initialize dp
    for (int i = 0; i < n*n; i++)
        dp[i] = 0;

    // Diagonal DP
    for (int len = 1; len < n; len++) {
        computeDP<<<1, n>>>(dp, d_dims, n, len);
        cudaDeviceSynchronize();
    }

    printf("Minimum cost: %d\n", dp[0*n + (n-1)]);

    cudaFree(dp);
    cudaFree(d_dims);

    return 0;
}

```

Writing nspdp.cu

```

In []: # The below command is used to compile the CUDA Program into executable file

# nvcc stands for NVIDIA CUDA Compiler.

# It is a compiler for GPU Programs and compiles .cu files into executable files.

```

In [19]:

```
!nvcc nspdp.cu -o nspdp
```

```
nvcc warning : Support for offline compilation for architectures prior to
'<compute/sm/lto>_75' will be removed in a future release (Use -Wno-deprecated-gpu-
targets to suppress warning).
```

In []:

```
# The below command is used to execute the executable file and fetch the output
```

In [20]:

```
!./nspdp
```

```
Minimum cost: 30000
```